Reflected XSS Vulnerability Report

Reflected XSS vulnerability found in the Search bar
Date: 1/3/26

---

1. Executive Summary

A Reflected Cross-Site Scripting (XSS) vulnerability was found in the search bar of the OWASP Juice Shop web application.

The vulnerability allows arbitrary JavaScript execution in the context of the application’s origin when malicious input is submitted via the search field.

Although session cookies are protected with the HttpOnly attribute, the vulnerability remains critical due to its ability to execute arbitrary client-side code and manipulate the DOM.

---

2. Vulnerability Details

A. Description

A Reflected Cross-Site Scripting (XSS) vulnerability exists in the search functionality of the application.

User-supplied input submitted via the search parameter is reflected back into the webpage without proper output encoding or sanitization. As a result, an attacker can inject malicious JavaScript that executes in the victim’s browser within the application's origin.

B. Affected Component

• Search bar (#/search)

C. Affected Endpoint

• GET /#/search?q=<user_input>
(q represents user-controlled input directly added into the DOM.)

D. Scope

- Target: OWASP Juice Shop
- Environment: Localhost (Docker deployment)
- Testing type: Manual Testing

E. Steps To Reproduce

• Navigate to the search bar
• In the search input field, enter the payload one by one:

<img src=x onerror=alert(1)>

• Submit the search request.
• Observe alert execution.

---

3. Proof of Concept (PoC)

- **Payload 1 – Execution Test**
  <img src=x onerror=alert(1)>
  Result:
  JavaScript executed successfully.
  Meaning:
  Confirms presence of XSS vulnerability.

- **Payload 2 – Application Context Confirmation**
  <img src=x onerror=alert(document.domain)>
  Result:
  Returned: localhost
  Meaning:
  Confirms that injected JavaScript executes within the application's origin context.

- **Payload 3 – Session Access Test**
  <img src=x onerror=console.log(document.cookie)>
  Result:
  Returned empty.
  Meaning:
  Session cookies are protected using the HttpOnly flag, preventing direct JavaScript access.

- **Payload 4 – Browser Information Extraction**
  <img src=x onerror=alert(navigator.userAgent)>
  Result:
  Returned browser and OS details.
  Meaning:
  Confirms access to client-side environment data and browser APIs.

---

4. Root Cause

The application reflects user-supplied search input directly into the DOM without applying proper contextual output encoding or sanitization.

The vulnerability likely arises from unsafe usage of DOM sinks such as

element.innerHTML = userInput

instead of secure rendering mechanisms.

---

5. Impact Assessment

Even though direct cookie theft is mitigated via HttpOnly, this vulnerability enables:

• Arbitrary JavaScript execution
• DOM manipulation
• Malicious redirection
• Browser fingerprinting
• Potential vulnerability chaining

An attacker could craft a malicious link containing the payload and trick a victim into clicking it, leading to execution within the trusted application context.

---

6. Remediation Recommendations

- Apply contextual output encoding to all user-controlled input.
- Avoid unsafe DOM sinks such as innerHTML with untrusted data.
- Use framework-provided sanitization features.
- Implement a Content Security Policy (CSP).
- Perform regular security code reviews and testing.

---

7. Conclusion

The application is vulnerable to XSS via the search functionality. While certain mitigations (HttpOnly cookies) reduce direct session theft risk, the vulnerability remains critical due to arbitrary JavaScript execution capability within the application's origin.

Immediate remediation is recommended.